INDIVIDUAL CAPSTONE REPORT

Title: **A Decentralized Freelance Protocol with Web3 Integration**

Date: 03/04/2026

April - 2026

Title: Decentralized Freelance Protocol with Web3 Integration

Date: 2026/04/03

General Information

Project Title: Decentralized Freelance Protocol with Web3 Integration

Points of Contact & Project Members

Position	Full Name	Student ID	Roles & Responsibilities (Brief)	E-mail
Supervisor	Subit Timalsina			
Team Lead	Sarun Mahrajan	0371759	Project Manager / Backend & Blockchain Developer	sarun2080-0228@iimscollege.edu.np
Member 1	Bijee Dangol	0372053	Frontend Developer	bijee2080-212@iimscollege.edu.np
Member 2	Pawan Poudel	0371478	Backend Database ,Security, Storage Specialist	pawan2080-0296@iimscollege.edu.np
Member 3	Anushree Pradhan	0372148	Backend, API Developer, Database Assistant	anushree2080-0344@iimscollege.edu.np
Member 4	Runa Maphu	0371373	System Workflow Engineer, Junior Backend Developer	runa2080-0189@iimscollege.edu.np

Contents

Chapter 1

Chapter one was already part of the proposal and is not included here.

Chapter 2

RELATED WORKS

2.1 Overview

The current gig economy is too centralized where platforms like fiverr, upwork, etc.. controls everything and all the data and decisions are in one place, this causes a single point of failure, if one of the systems break everything goes down and platform also takes huge fees for the work, the first solution was to fully decentralize the system where people tried using blockchain to remove central control which improved transparency where everything is visible and transparent but with this came new problems such as high gas fees (expensive transactions) and bad user experience (hard to use), but the current idea, instead of choosing one of two we combine both and create a hybrid model where we use centralized system for speed, coordination and easier user experience and we use decentralized system for security, transparency and trust.

2.2 Review of Previous Work

2.2.1 Decentralized Storage and Data Integrity

One of the main topics of current storage study is the problem of ”blockchain bloating.” Blockchain is not very good for storing large data because if you store everything on the blockchain, nodes have to handle too much data which causes

performance drop and slows down the whole system, hence blockchain becomes heavy and inefficient the solution is to use IPFS and blockchain together, instead of storing full data on blockchain, store data in IPFS (off-chain storage) and on the blockchain only store Content ID (CID) (Zhu et al., 2026) which results in storage reduction by 75%, keeps the blockchain lightweight and still secure.

The study by Athanere (2022) supports (Zhu et al,. 2026) by demonstrating a semi-decentralized design that uses a mix of local processing by hashing the data locally and peer to peer storage which keeps global blockchain small, while maintaining data security and integrity and faster access compared to fully decentralized system. furthermore, the study of Tereshchenko (2025) discusses how finding data in distributed networks can be slow and how searching via hash tables is not always efficient. The solution is to use a relational database to index hashes which improves data retrieval speed and works efficiently even with large datasets.

Although these studies improve storage scalability and data security, the problem of effective data retrieval in large distributed system remains unresolved. Although indexing mechanisms helps to improve performance, but it is not fully integrated with the whole system that is the integration of off-chain storage (IPFS), relational databases and real time application needs, so this highlights a gap in designing a unified system that ensures both efficient storage and fast data access.

2.2.2 Automation and Trust in Freelancing

One of the primary concepts in recent blockchain articles is the automation of professional trust. According to Buterin (2014) Ethereum blockchain models which uses public ledger for transparent records and smart contracts for automatic rules removes the need for intermediaries leading to less fraud, removing the need of central authority and trust is handled by code. According to the study Luo (2019) smart contracts reduce disputes because smart contract works using “if-then “logic (i.e, if work is submitted and verified only then the payment will be released) in this way the payment happens automatically, less need for manual dispute handling and faster resolution.

In the case of professional identification, (Lim, 2021) found that in traditional platforms your reputation has lock- effect where your reputation can’t carry on to another platforms, but with blockchain your skills and history are stored on-chain meaning your reputation is verifiable and can carry it across platforms which results in more freedom for freelancers and less dependency on one platform. Guan (2023) supports this by pointing out that hybrid designs match the high-fidelity user experience of traditional web systems with the transparency of distributed

ledger technology, making them more sustainable for Web3 adoption.

Although these studies show that blockchain can improve trust, transparency, and reputation management in freelance platforms, they still fail to provide a clear path for addressing the practical challenges in the real world. For instance, the freelancer reputation system based on blockchain mainly aims at promoting a freelancer's trust but overlooks important aspects such as system scalability and enhancing user experience. similarly, Guan (2023) highlights even bigger challenges to the Web3 adoption including restrictions on usability and integration. While Buterin (2014) defines a trustless system from the conceptual standpoint, it ignores the issues at the application level such as performance and system coordination side. This highlights a gap in designing a practical system that accounts for trust, usability, and decentralization freelancing platforms scalability.

2.2.3 Backend Performance and Security

In studying the effectiveness of web backends in distributed scenarios, Bednarz (2025) discovered that in distributed systems like blockchain apps, some operations are slow, example, waiting for blockchain confirmation in doing so other users using the app gets delayed because the operation blocks threads, and the solution he found that using asynchronous processing (async loops) are necessary to prevent thread starvation during slow external events, such as waiting for blockchain confirmation. According to his methodology asynchronous frameworks can handle thousands of simultaneous users while maintaining low latency. Stonebraker (2010) according to his study the performance bottlenecks in traditional systems are because of bad system design rather than sql databases itself. If designed properly relational databases are capable of handling large scale workloads as well as high speed transactions.

Lastly, the study by Amuthadevi et al. (2022) demonstrate that by enforcing security measures early in the development cycle we can mitigate web vulnerabilities. The study recommends implementing 10 levels of "salting and hashing" for password storage and using middleware for API's so we can control who can access what, which prevents data leaks and protects sensitive informations. Furthermore Huang et al. (2020) suggests that for the data integrity we use blockchain as a tracking system for files but we store metadata on blockchain, because blockchain is secure but inefficient for large data, so we use it to verify data, not store it.

Although these studies highlight important advancement Bednarz (2025) focuses on performance benchmarking of web frameworks, Stonebraker (2010) discusses database architecture efficiency without considering decentralized system integration. Amuthadevi et al. (2022) focuses on early security implementations

but do not address with distributed systems

(IPFS/blockchain), additionally Tereshchenko et al. (2025) focuses on improving data retrieval through indexing but do not explore synchronization with blockchain based systems. Hence, while these studies all work separately, they do not demonstrate these components can be integrated to create a unified system.

2.3 Conclusion of Literature Review

According to the reviewed studies, the industry standard for decentralized applications is a hybrid model that combines IPFS, Blockchain and efficient backend logic to make everything fast, secure and scalable. Authors frequently identified that off-chain (IPFS) storage reduces blockchain sizes which saves storage while blockchain handles payments using smart contracts which effectively automates trustless payments, the literature also highlights that even with IPFS and blockchain working together the data retrieval from peer to peer network is slow hence asynchronous backend logic and relational indexing are essential to overcome the latencies.

2.4 Literature Review Summary

Table 2.1: Literature Review Summary

Authors & Year	Methodology	Key Findings	Gap / Contribution
Zhu et al. (2026)	IPFS- BC collaborative storage model.	75% reduction in blockchain storage consumption.	increases storage scalability but ignores system integration and data retrieval delay.
Stonebraker (2010)	Deconstruction of DB overhead (Locking/Logging).	Performance bottlenecks are architectural, not language based.	ignores decentralized system integration in favor of concentrating on conventional databases
Amuthadevi et al. (2022)	OWASP analysis and E-Commerce secure testbed.	Middlewares and 10-level hashing prevent data exposure.	ignores compatibility with distributed storage systems in favor of security.
Athanere (2022)	Hierarchical semi-decentralized design.	Local hashing and pinning keep the global ledger fast.	achieves a balance between efficiency and security, but it ignores issues with large-scale data retrieval.
Tereshchenko et al. (2025)	Hybrid storage with indexing	Improved data retrieval performance using relational indexing in distributed environments	Does not address real-time synchronization between database and blockchain systems
Buterin (2014)	Ethereum smart contract model	Enables trustless interactions through smart contracts and public ledgers	lacks details about implementation for performance and scalability but offers a theoretical basis
Luo (2019)	Smart contract milestone logic.	"If-then" logic saves time in dispute resolution.	overlooks system performance and scalability to the benefit of payment automation.
Lim (2021)	Statistical analysis of on-chain reputation.	Verified skills minimize fraudulent reviews and lock- in.	ignores system integration and backend performance in favor of identification.
Bednarz (2025)	Load testing on Python frameworks.	Asynchronous loops prevent thread starvation during I/O.	ignores blockchain and distributed system interaction in favor of backend performance.
Huang et al. (2020)	Secure file-sharing with IPFS proxy.	Blockchain acts as a trackable server for metadata.	enhances data integrity but overlooks system coordination and retrieval efficiency.

Chapter 3

SYSTEM ANALYSIS

3.1 Theoretical SWOT Analysis

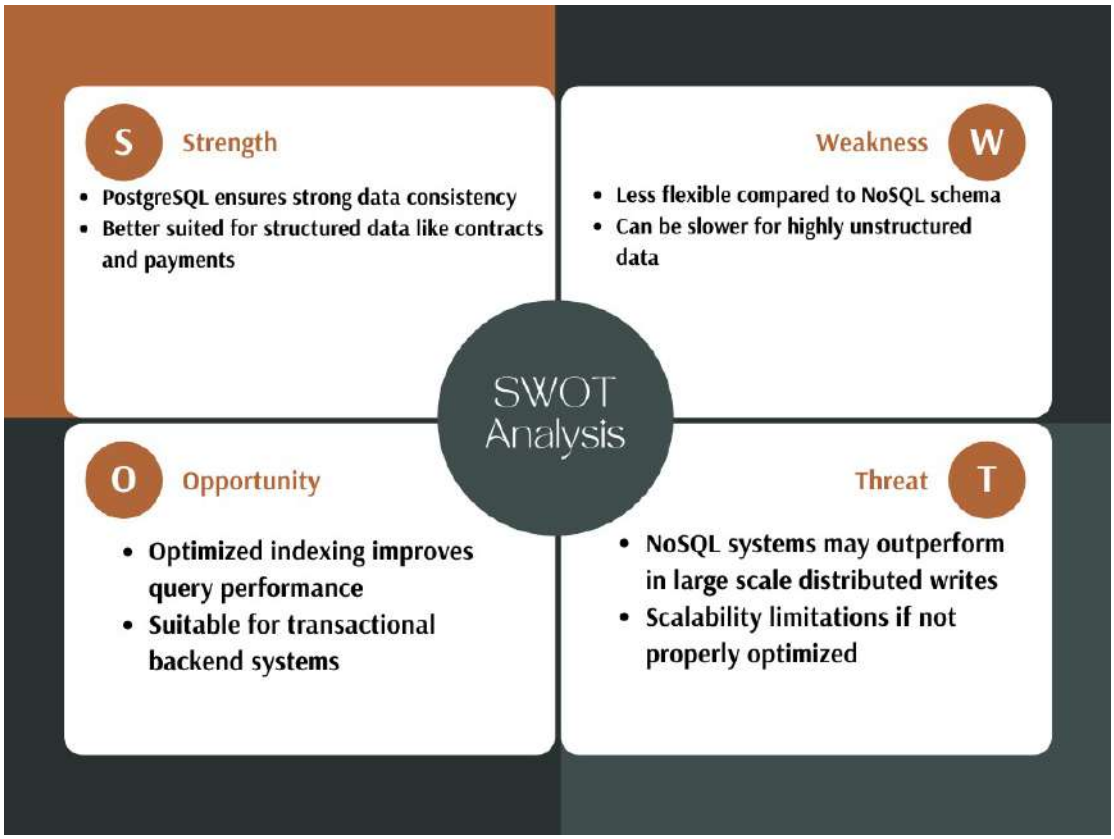


Figure 3.1.1: SWOT Analysis: PostgreSQL vs NoSQL

This SWOT analysis indicates that PostgreSQL's excellent data consistency and transactional operation capability make it a better fit for the system, while

NoSQL databases, like MongoDB offer better flexibility, they don't have strict relational integrity which is essential for handling payments and contracts. Therefore, PostgreSQL is selected to ensure reliable and consistent data handling

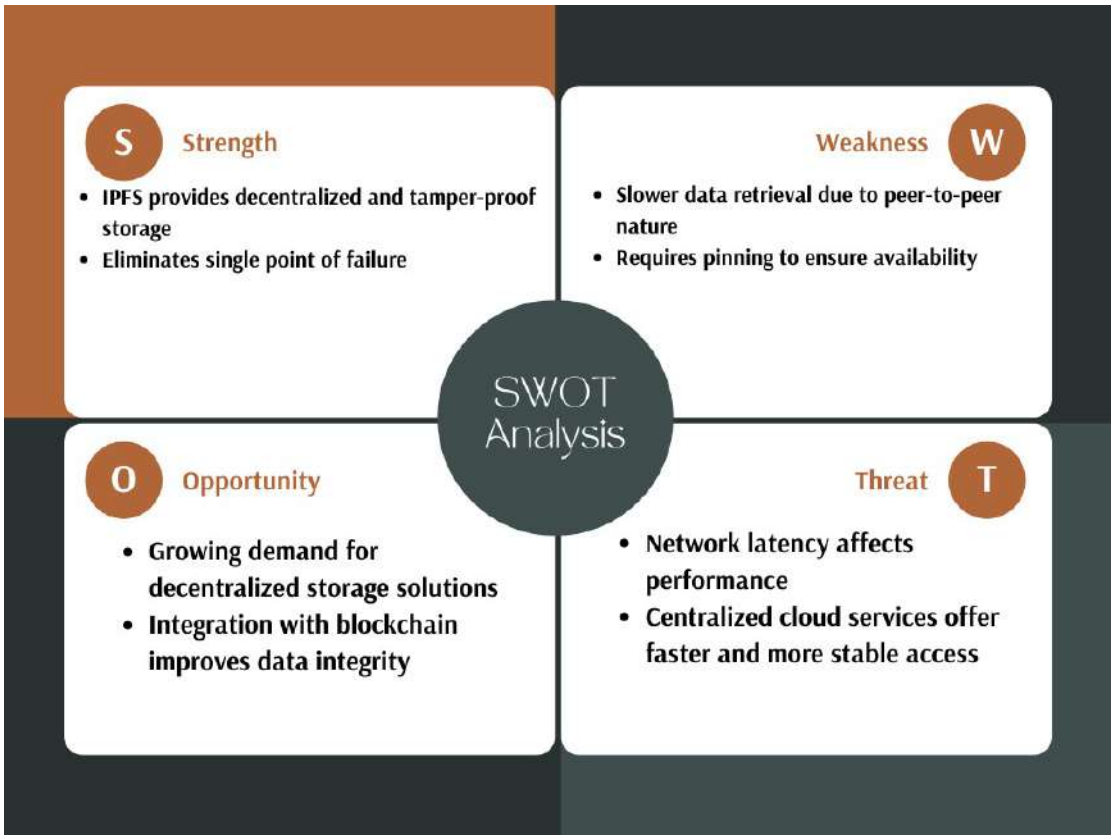


Figure 3.1.2: SWOT Analysis: IPFS vs Cloud Storage

This swot analysis shows that IPFS is a better fit for the system because it eliminates the dangers associated with centralized cloud services and offer decentralized tamper proof storage, although cloud storage offers fast and more stable access, but it also introduces a single point of failure, therefore IPFS is selected to guarantee data availability and integrity in a decentralized environment.

3.2 Technical Analysis and Justification

The following technologies were selected based on their ability to fulfill the hybrid system requirements of security, scalability, and performance. And efficient integration between centralized and decentralized components

3.2.1 PostgreSQL

PostgreSQL is preferred over NoSQL for the following reasons:

- PostgreSQL can be used for off-chain relational database and to store meta-data.
- PostgreSQL ensures consistency in tracking contracts, user actions and milestones
- PostgreSQL is ACID compliant which help us prevent issues such as incorrect payment or state mismatches

PostgreSQL is preferred compared to NoSQL because PostgreSQL provide stronger data integrity for structured relationships

3.2.2 IPFS (InterPlanetary File System)

IPFS is preferred over Centralized Cloud storage for the following reasons:

- IPFS can be used as a decentralized storage for project deliveries
- IPFS generates a unique CID for every file uploaded
- IPFS prevents tampering and ensures data integrity

IPFS is preferred compared to Cloud storage because cloud storages (for example AWS S3) or centralized storages have single point of failure since everything is centralized but since IPFS is p2p (peer to peer) network there are no single point of failure.

3.2.3 Blockchain (Ethereum Smart Contracts)

Why smart contracts (blockchain) is preferred compared to centralized systems:

- Blockchain can be used to automate milestone based payment by using predefined rules.
- Blockchain is transparent and public ledger of all interactions and removes the need for middlemen/intermediaries.
- Blockchain also ensures that all transactions are secure and verifiable.
- Blockchains can also be used to handle payments and contract agreements.

Blockchain is preferred compared to other centralized systems, centralized payment gateways like paypal or upwork are opaque and charge high fees in percentages (5% to 20%) for each transactions while blockchain fees are not percentage based instead they depend on network conditions and transaction complexity, this makes blockchain more transparent.

3.3 Requirement Specifications

3.3.1 Functional Requirements

The functional requirements according to my role of Backend, database, storage and security specialist:

Data Storage and management

- The project metadata, milestones and user related data must be all stored in PostgreSQL by the system
- The consistency between stored data and system states must be maintained by the system

Decentralized file storage

- Users should be able to upload project files or files related to project to IPFS
- Content Identifier (CID) must be generated by the system for each uploaded file
- The system must link uploaded files to corresponding milestones.

Data Synchronization

- The system must sync all the data across PostgreSQL, IPFS and blockchain
- The system must ensure that whenever something gets updated it must be updated everywhere correctly

Backend Processing

- The backend must handle user requests and coordinate communication between database, IPFS and blockchain.
- To handle delays from blockchain and ipfs operations the system must support asynchronous processing

Security and access control

- To restrict unauthorized data access the system must have role based access control
- All the incoming request must be validated by the system before processing it.

3.3.2 Non-Functional Requirements

the non functional requirements are as follows:

Performance

- the database queries must be executed with low latency.
- multiple concurrent requests must be handled buy the system without blocking operations.

Security

- sensitive data must be protected using salting and hashing techniques.
- To enforce secure API access middleware must be implemented.

Scalability

- Blockchain storage must be reduced by storing only CID on IPFS.
- The backend must support scaling with user and data loads.

Availability

- To ensure the availability of the files stored in IPFS those files must be pinned.
- Even if individual nodes were to fail the system must remail operational.

Reliability

- The data must be consistent and maintained between off -chain and on-chain components
- System failures must not result in data loss or inconsistency.

Chapter 4

SYSTEM DESIGN

4.1 Sequence Diagrams

This section presents the essential sequence diagrams relevant to the backend, database, and se- curity roles. They illustrate the interactions among the FastAPI backend, PostgreSQL database, IPFS distributed storage, and Ethereum blockchain.

4.1.1 MetaMask Authentication Sequence Diagram

In this Sequence diagram a freelancer/client will first connect to wallet then the system sends nonce where the user sings the message with his/her cryptographic hash and they system will verify if the hash is vaild then it will create a session, and in the case the verification fails the session will not start and the user will not be authenticated

In the diagram the nonce is stored in the postgreSQL, where backend server sends the temp nonce and the postgreSQL verifies it, if the nonce matched the user would start authenticating.

4.1.2 Contract Creation Sequence Diagram

In this sequence diagram a freelancer/client creates project/contract after creating the metadata would be saved in PostgreSQL and the contract details will be stored in IPFS and it would return CID, The CID would be linked to database which will be helpful later if we want to find the files we want. Then the process

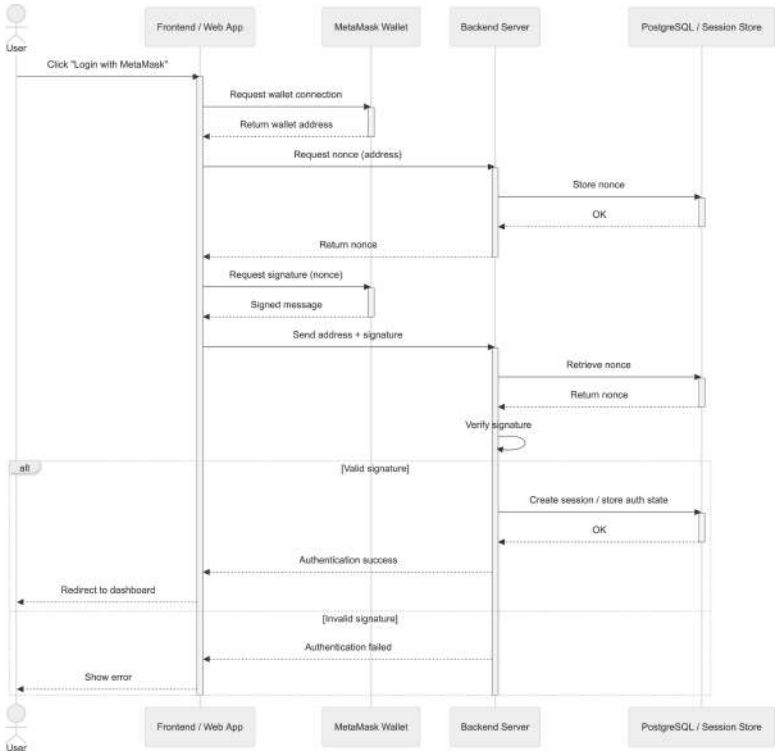


Figure 4.1.1: MetaMask Authentication Sequence Diagram

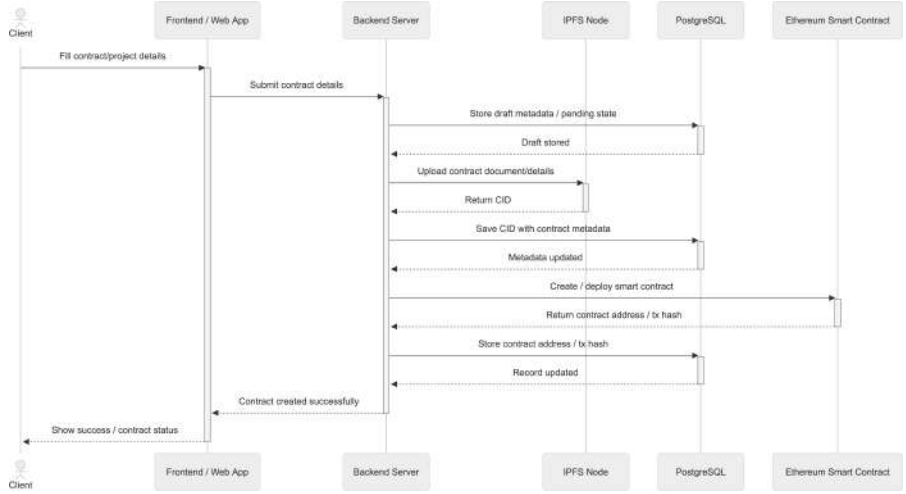


Figure 4.1.2: Contract Creation Sequence Diagram

of creating a smart contract will start on the blockchain , the blockchain will return contract transaction hash and its record will be updated to the backend server and the client will receive the success message.

4.1.3 Milestone Submission Sequence

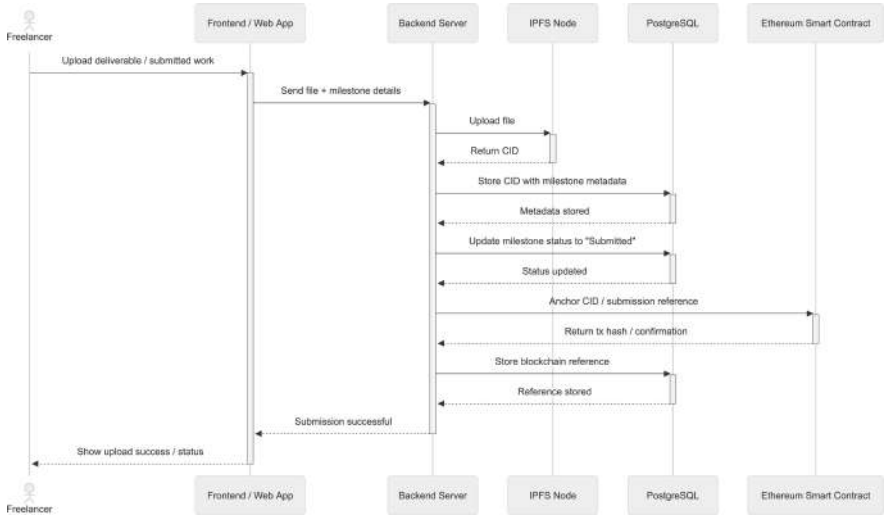


Figure 4.1.3: Milestone Submission Sequence

In this sequence diagram the freelancer/client will first upload their work and the work will be stored in the IPFS which then returns CID afterwards, CID and milestone metadata are saved in PostgreSQL after this the submission status is updated and the submission reference, transaction hash are stored in the backend server. Then the client will get the success message

4.1.4 Milestone Approval and Payment Release

In this sequence diagram the client reviews the work submitted by the freelancer he hired and he approves the work from frontend then the approval is recorded in the backend server which then PostgreSQL verifies the milestone status after successful verification it will trigger payment release (smart contract) blockchain returns transaction confirmation/ tx hash to the backend server afterwards, the milestone status will be updated to “paid” then the transaction reference will be stored and the payment will be released and the client will see payment confirmation.

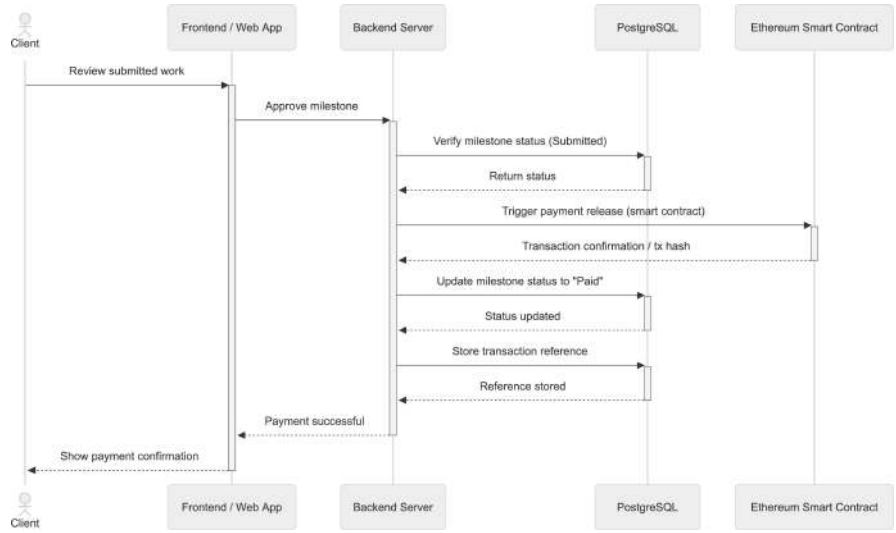


Figure 4.1.4: Milestone Approval and Payment Release

4.1.5 Dispute Initiation Sequence Diagram

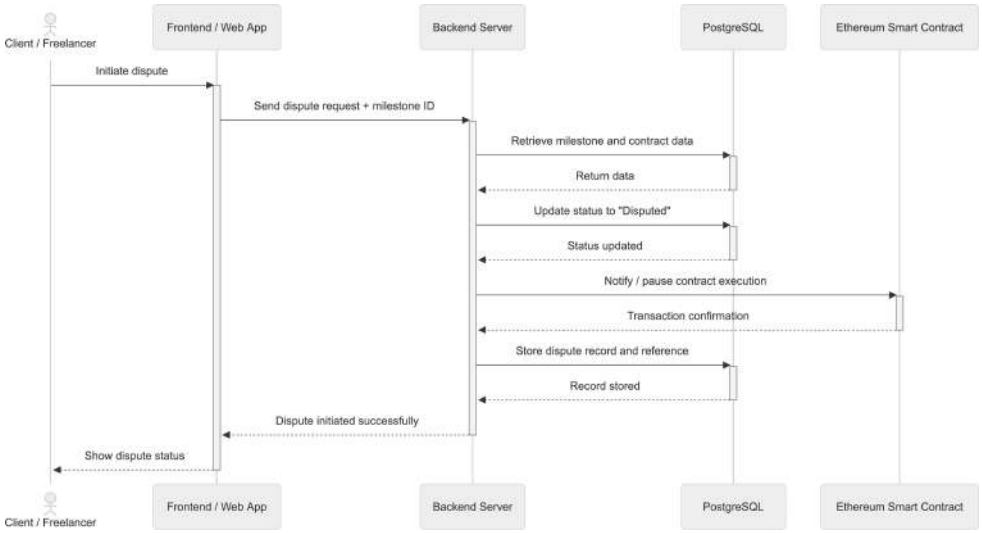


Figure 4.1.5: Dispute Initiation Sequence Diagram

In this sequence diagram the client or freelancer raises dispute , the dispute is recorded by the system and stored in PostgreSQL afterwards contract or deliverables are retrieved related to dispute then the status us updated to dispute, and some admin will be assigned to oversee the dispute.

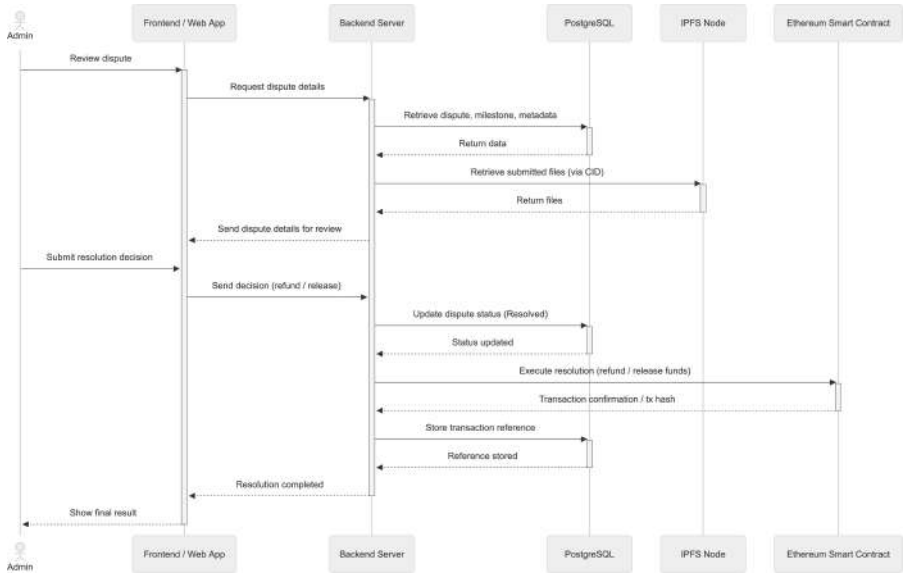


Figure 4.1.6: Dispute Resolution Sequence Diagram

4.1.6 Dispute Resolution Sequence Diagram

In this sequence diagram admin will oversee the dispute between a freelancer and client , the admin will request the dispute details from backend server and the system fetches metadata from postgreSQL and files from IPFS and sends the details to admin for review after the decision has been made blockchain will execute refund/release and the status will be updated “resolution completed”

4.2 Activity Diagrams

4.2.1 MetaMask Authentication Activity Diagram

In this activity diagram, it illustrates the process of user authenticating using metamask where the user clicks login and connects to their metamask wallet then the system retrieces the user wallet address, the backend then generates a nonce and stores it in the database, the user then signs the nonce using metamask, afterwards signed message is sent to the backend for verification after backend verifies the signature to confirm identify, if the identity is confirmed a session is

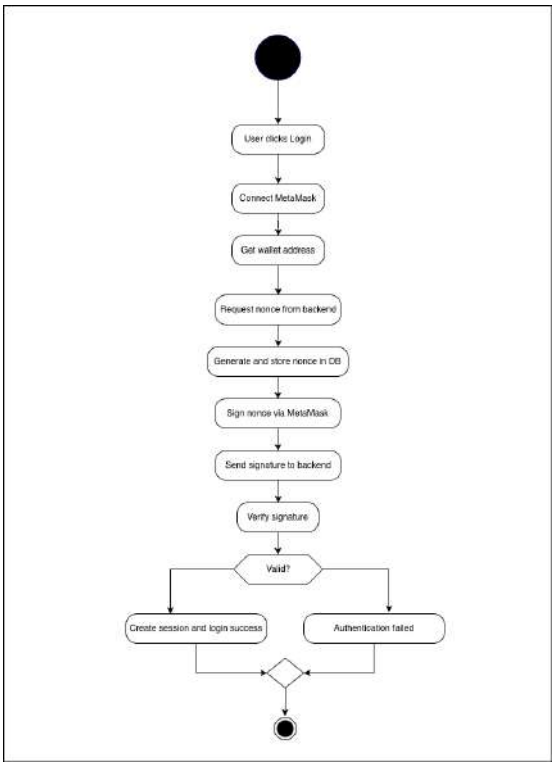


Figure 4.2.1: MetaMask Authentication Activity Diagram

created and login successful, if identity is invalid then the authentication fails

4.2.2 Contract Creation Activity Diagram

In this activity diagram, it illustrate the process of contact creation and storage, when client enters contract details and submits them to the backend, the back-end validates and stores contract metadata PostgreSQL , then contract data is uploaded to IPFS for decentralized storage, afterwards IPFS generates a CID to uniquely identify the stored data then the backend stores the CID and creates smart contract on the blockchain, then the contract is addressed or transaction hash is saved in PostgreSQL, then contract is successfully created and ready to use.

4.2.3 Milestone Submission Activity Diagram

In this diagram, when freelancer uploads work and sends it to the backend the file is uploaded to IPFS for decentralized Storage and IPFS generates a CID for the uploaded file then the CID is stored in PostgreSQL by the backend, the milestone status will be updated to “Submitted” then CID is anchored to the blockchain

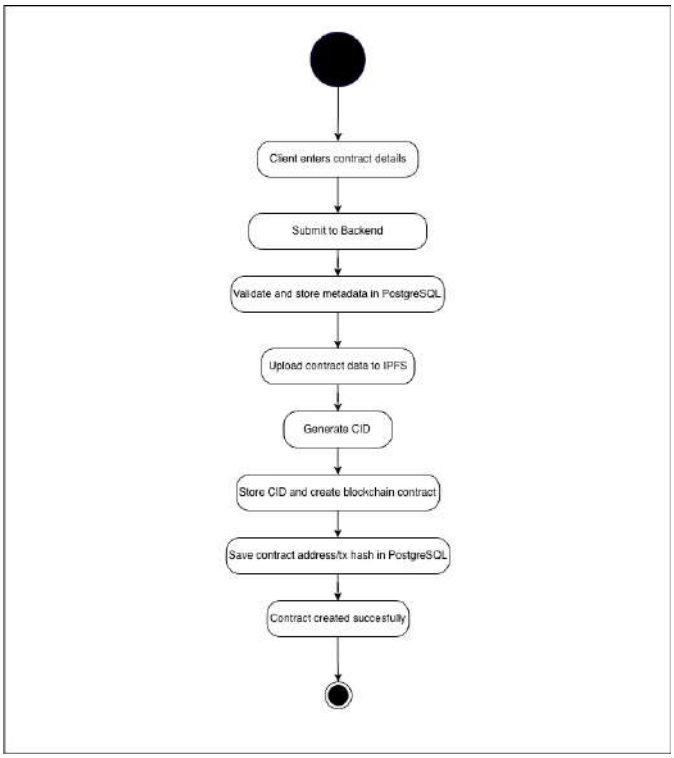


Figure 4.2.2: Contract Creation Activity Diagram

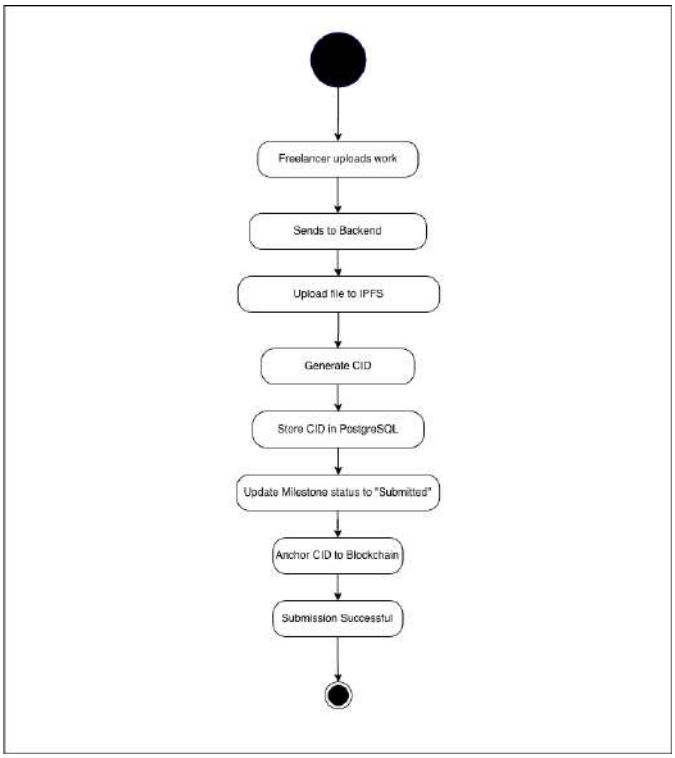


Figure 4.2.3: Milestone Submission Activity Diagram

for verification and submission is completed successfully

4.2.4 Milestone Approval Activity Diagram

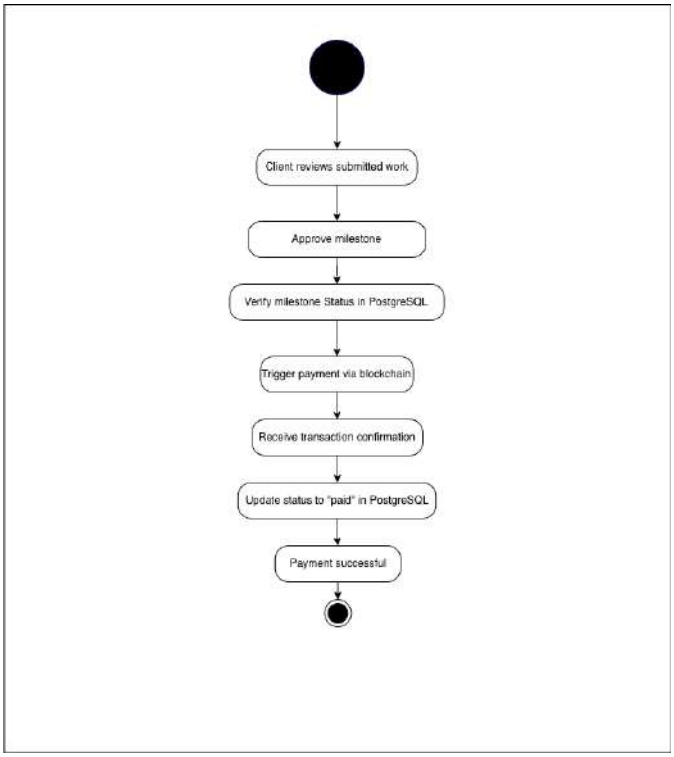


Figure 4.2.4: Milestone Approval Activity Diagram

In this diagram , it illustrates the process of approval and payment release, it starts when client reviews submitted work and if its all ok then the client will approve the milestone afterwards the backend verifies milestones status in the database then the payment is triggered through the blockchain and transaction will go through and the client will receive transaction confirmation and the milestones status is updated to “paid” in PostgreSQL and payment process is completed.

4.2.5 Dispute Initiation Activity Diagram

In this diagram, when a user either freelancer or client initiate dispute, the user’s request is sent to the backend after backend retrieves the milestones data from PostgreSQL the status of milestone will be updated to “Disputed”. The details of the disputes will be recorded in the database and the blockchain is notified

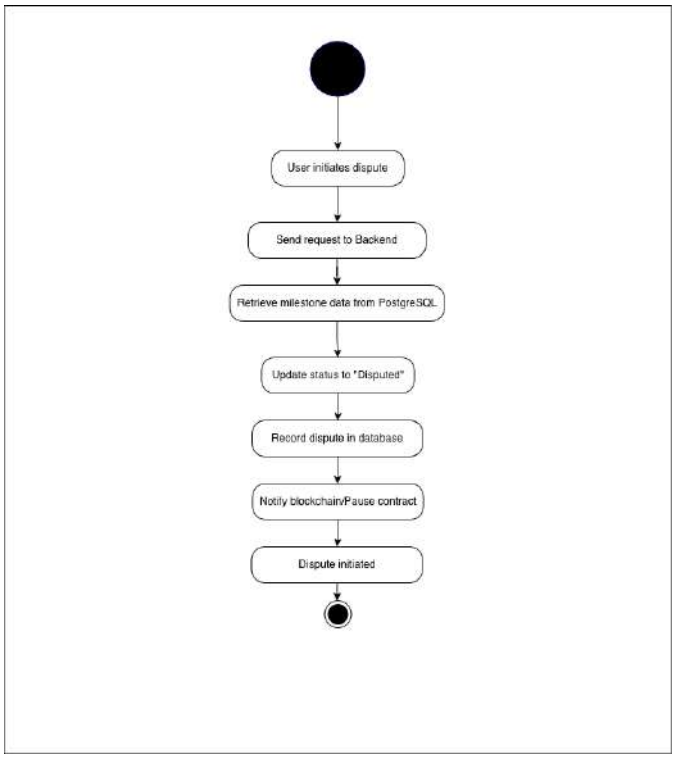


Figure 4.2.5: Dispute Initiation Activity Diagram

and the blockchain will pause the contract and with this the dispute process is initiated.

4.2.6 Dispute Resolution Activity Diagram

In this diagram it is shown how a dispute is handled and resolved by a admin. First when the admin is notified about the dispute the admin will review the dispute, relevant data is retrieved from PostgreSQL, the associated files related to the dispute is retrieved from IPFS, after the admin checks the milestones and the work done by the freelancer and based on the results the admin makes a decision to either refund the client or release the payment to freelancer, accordingly the blockchain executes refund or payment and system updates status in PostgreSQL and the process of dispute resolution is completed successfully.

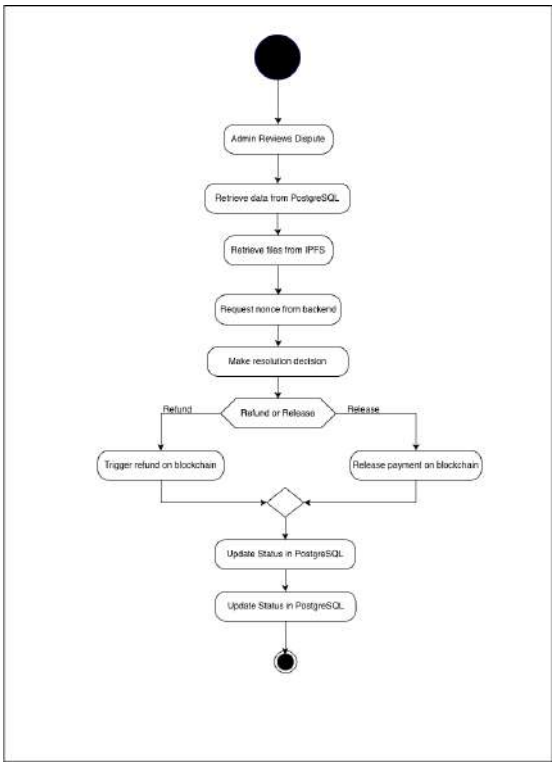


Figure 4.2.6: Dispute Resolution Activity Diagram

4.3 System Architecture Diagram

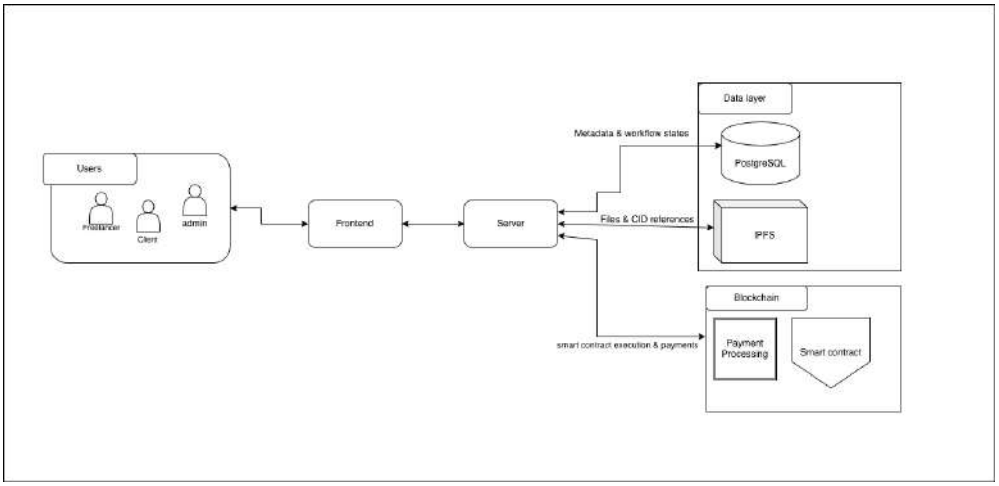


Figure 4.3: System Architecture

This system architecture demonstrates how the frontend, which interacts with the backend server, allows users to engage with the platform. All operations are coordinated and system logic is managed by the backend. It uploads files to IPFS, where they are referenced via CIDs, and keeps metadata and process statuses in

PostgreSQL. In order to carry out smart contracts and handle payments, the backend also communicates with the blockchain.

4.4 Class Diagram

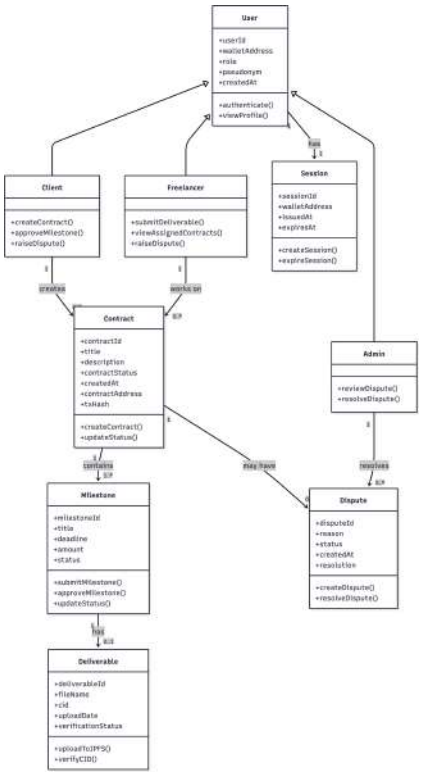


Figure 4.4: Class Diagram

The fundamental components of the decentralized freelance platform and their connections are shown in the class diagram. Client, Freelancer, and Admin all have unique responsibilities and functions within the system, and the User class serves as their foundation. Clients create contracts, which freelancers work on. Each contract has several milestones connected to deliverables that are kept using CIDs. The administrators can step up and settle disputes if there some thar arise, ensuring fair results. Sessions are also used to manage system access and user authentication within the system.

4.5 Pseudocode

4.5.1 Wallet Authentication Logic

```
# Authenticate user via signed message from Meta Mask
FUNCTION verify_wallet_signature(address, signed_message,
   original_hash):
    recovered_address = recover_public_key(signed_message,
       original_hash)
    IF recovered_address EQUALS address:
        session = create_new_session(address)
        RETURN session
    ELSE:
        RETURN error("Invalid Signature")
```

4.5.2 Milestone Submission Workflow

```
# Process a deliverable submission asynchronously
ASYNC FUNCTION submit_milestone(milestone_id, file):
    # Step 1: Upload to IPFS
    cid = AWAIT ipfs_client.add(file)
    # Step 2: Record CID on Blockchain
    tx_hash = AWAIT contract_interface.record_submission(
       milestone_id, cid)
    # Step 3: Wait for confirmation and update PostgreSQL
    AWAIT blockchain_confirm(tx_hash)
    database.update_milestone(milestone_id, status="Submitted
       ", cid=cid)
    RETURN success
```

4.5.3 Escrow Release Logic

```
# Release payment to freelancer upon client approval
ASYNC FUNCTION approve_and_pay(milestone_id):
    # Verify current status in DB
    IF database.get_status(milestone_id) == "Submitted":
        # Trigger Smart Contract payout
        tx_hash = AWAIT contract_interface.release_funds(
           milestone_id)
```

```
        AWAIT blockchain_confirm(tx_hash)
        database.update_milestone(milestone_id, status="Paid")
        RETURN success
    ELSE:
        RETURN error("Invalid State Transition")
```

Chapter 5

REFERENCES

[1] Amuthadevi, C., Srivastava, S., Khatoria, R., & Sangwan, V. (2022). A study on web application vulnerabilities to find an optimal security architecture. SRM Institute of Science and Technology.

[2] Athanere, S. (2022). Blockchain-based hierarchical semi-decentralized approach using IPFS for secure and efficient data sharing.

[3] Bednarz, B. (2025). Benchmarking the performance of Python web frameworks.

[4] Buterin, V. (2014). Ethereum: A next-generation smart contract and decentralized application platform. https://ethereum.org/en/whitepaper/

[5] Guan, C. (2023). An ecosystem approach to Web 3.0: A systematic review and research agenda. Journal of Electronic Business and Digital Economics.

[6] Huang, H.-S., Chang, T.-S., & Wu, J.-Y. (2020). A secure file sharing system based on IPFS and blockchain. In Proceedings of the 2nd International Electronics Communication Conference.

[7] Lim, J. (2021). IT freelancer matching with exploiting blockchain in the gig economy.

[8] Stonebraker, M. (2010). The NoSQL discussion has nothing to do with SQL. Communications of the ACM, 53(4).

[9] Tereshchenko, G., Kyrychenko, I., Vysotska, V., Hu, Z., Ushenko, Y., & Talakh, M. (2025). Hybrid system for image storage and retrieval in big data environments. I.J. Image, Graphics and Signal Processing, 17(3), 55–84.

[10] Zhu, T., Huang, Y., Liang, Z., Qin, M., Niu, R., Ma, Y., & Feng, Q. (2026). Research on an on-chain and off-chain collaborative storage method based on blockchain and IPFS. Future Internet, 18(2).

Chapter 6

APPENDIX

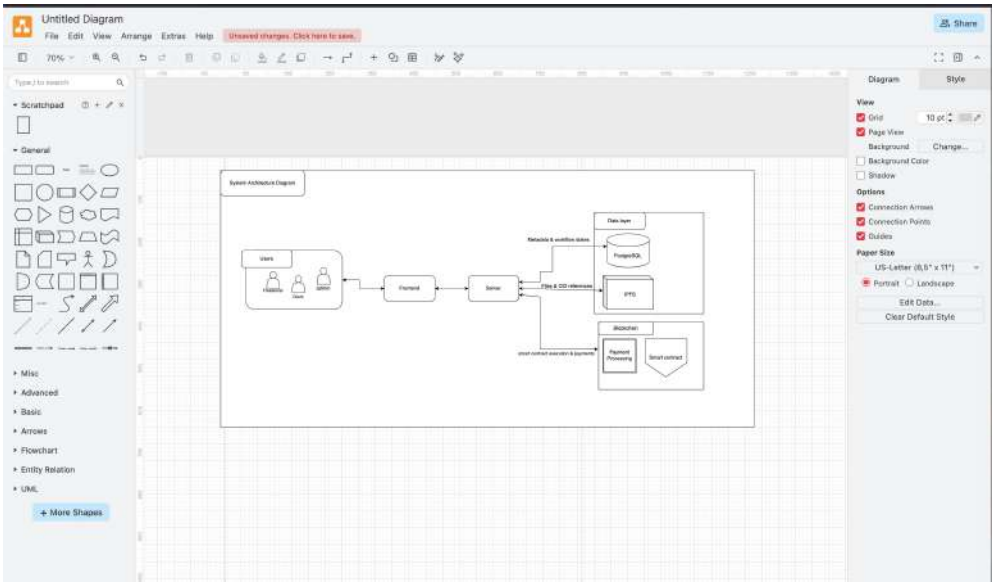


Figure 6.1.1: Appendix: System Architecture

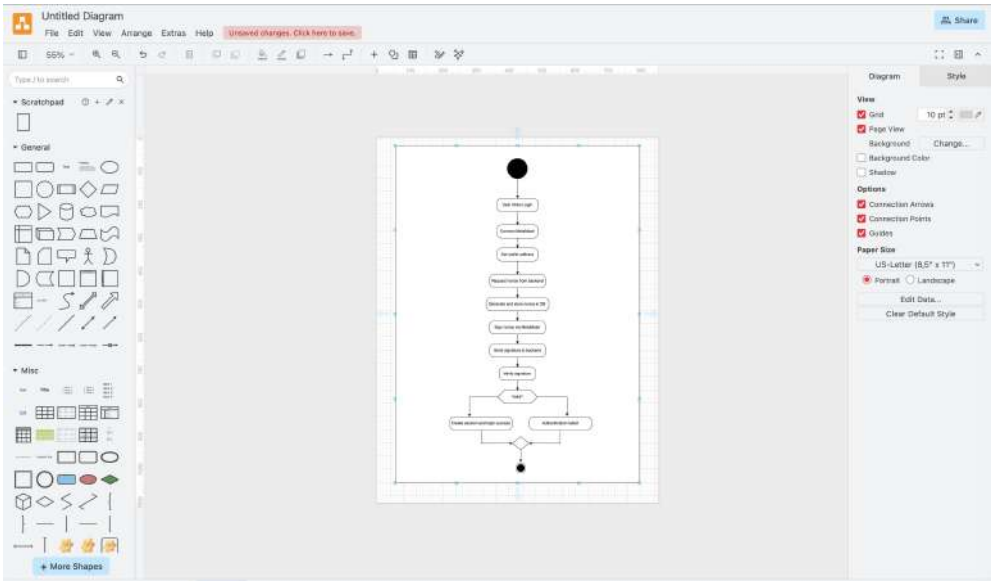


Figure 6.1.2: Appendix: MetaMask Activity

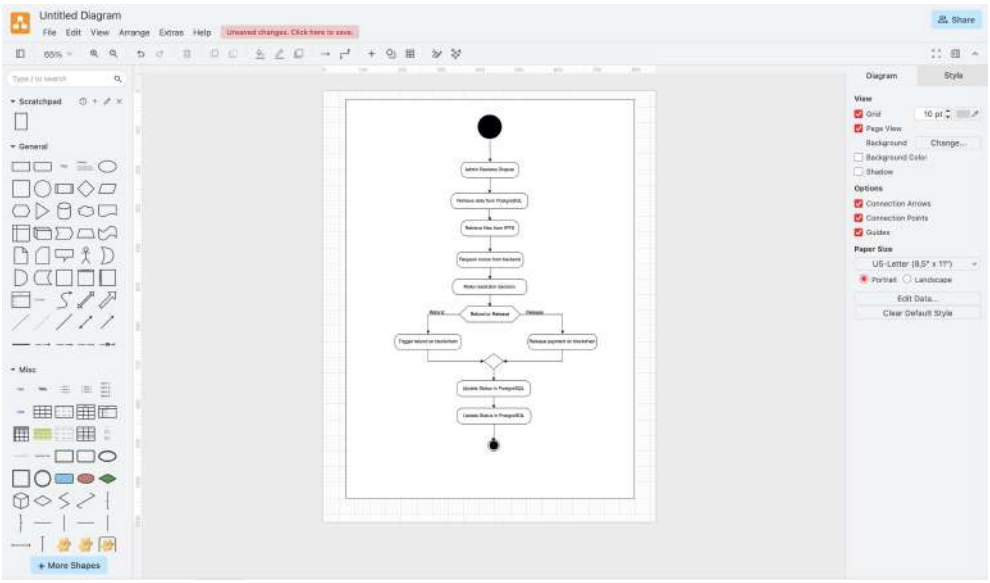


Figure 6.1.3: Appendix: Dispute Resolution

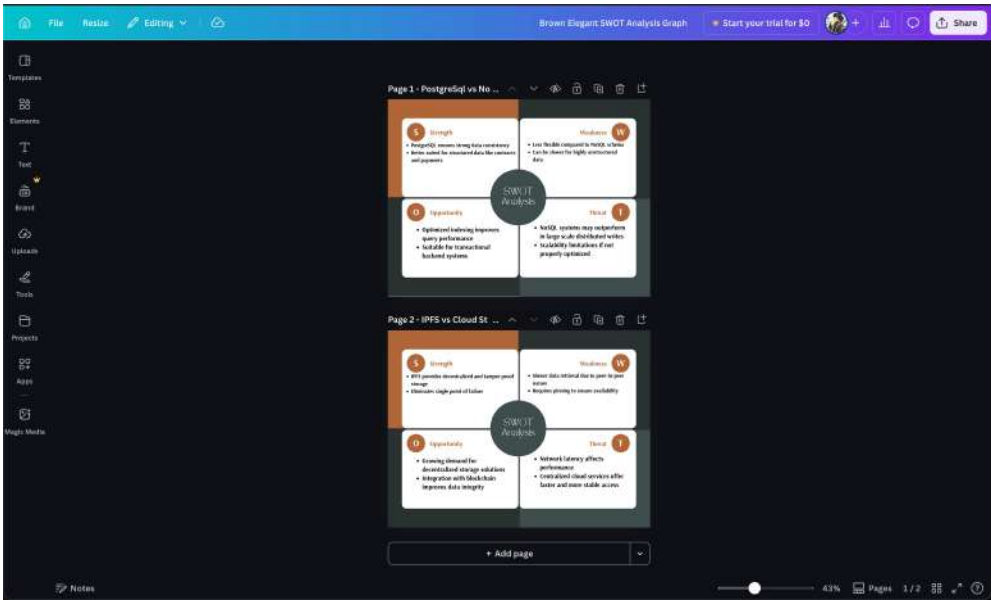


Figure 6.1.4: Appendix: SWOT Analysis

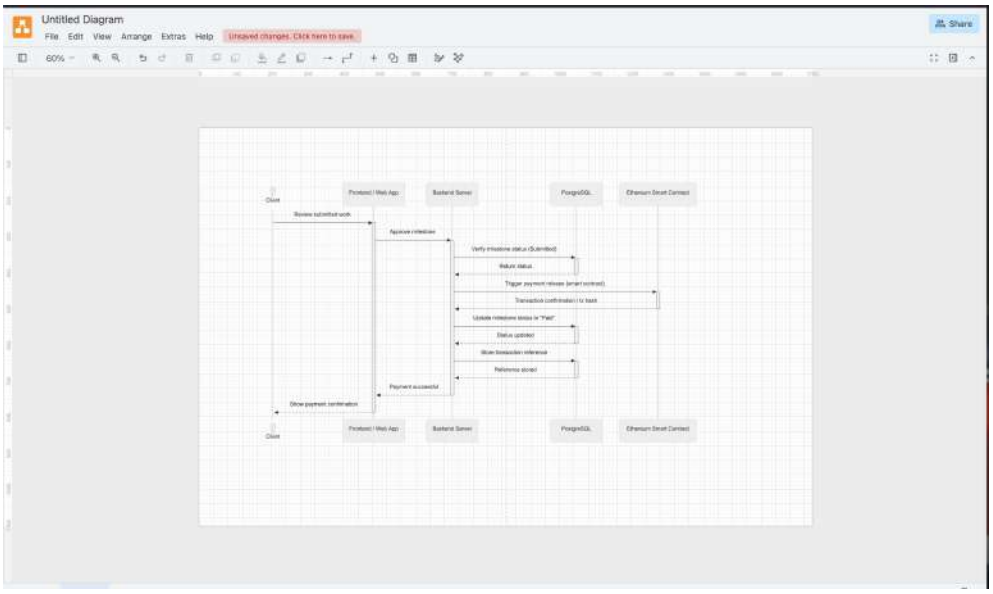


Figure 6.1.5: Appendix: Sequence Diagram

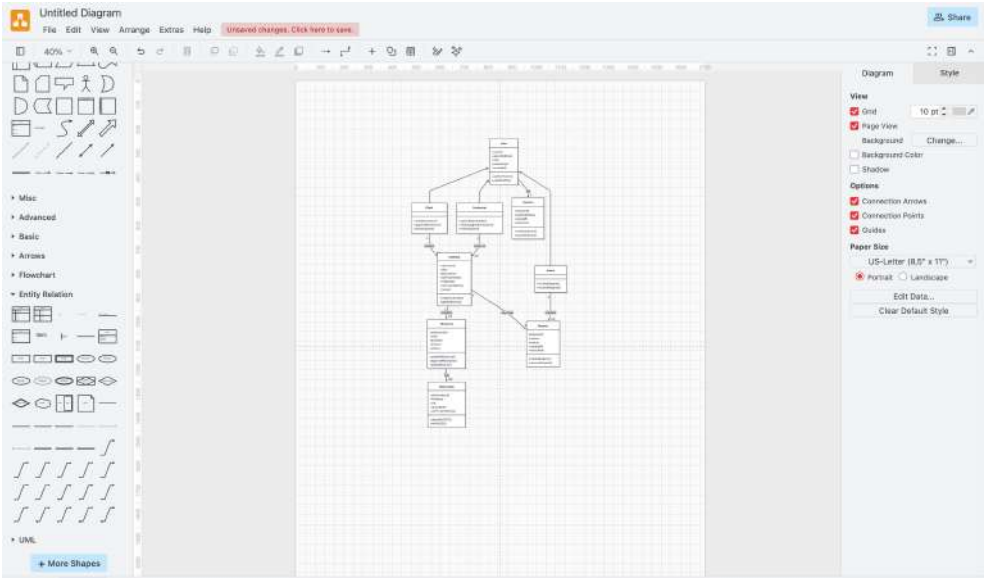
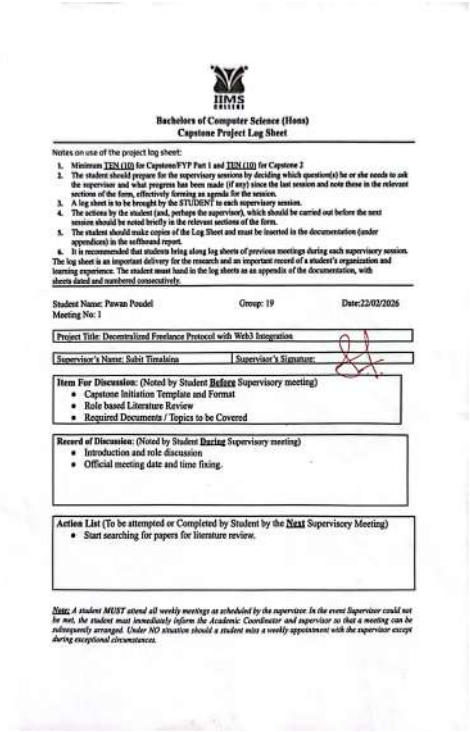


Figure 6.1.6: Appendix: Class Diagram

6.2 Log Sheets



Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum TEN (10) for Capstone/FYP Part 1 and TEN (10) for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.

The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Pawan Poudel | Group: 19 | Date:22/02/2026
Meeting No: 1

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalsina | Supervisor's Signature:

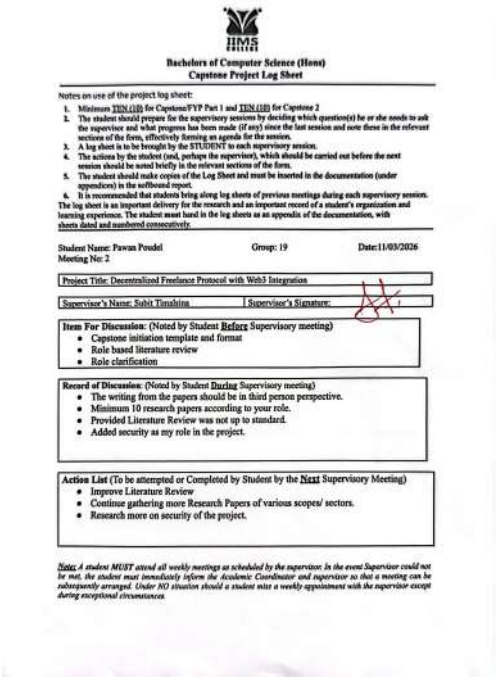
Item For Discussion: (Noted by Student **Before** Supervisory meeting)
- Capstone Initiation Template and Format
- Role based Literature Review
- Required Documents / Topics to be Covered

Record of Discussion: (Noted by Student **During** Supervisory meeting)
- Introduction and role discussion
- Official meeting date and time fixing.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)
- Start searching for papers for literature review.

Note: A student MUST attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.

Figure 6.2.1: log sheet 1



Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum TEN (10) for Capstone/FYP Part 1 and TEN (10) for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.

The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Pawan Poudel | Group: 19 | Date:11/03/2026
Meeting No: 2

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalsina | Supervisor's Signature:

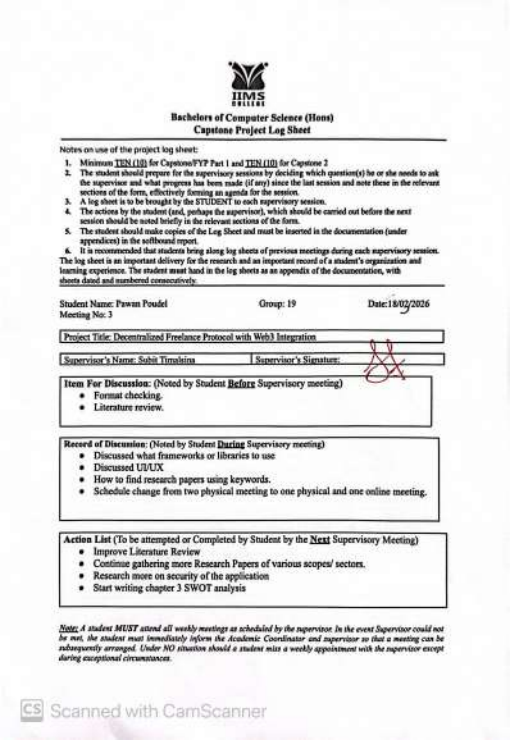
Item For Discussion: (Noted by Student **Before** Supervisory meeting)
- Capstone initiation template and format
- Role based literature review
- Role clarification

Record of Discussion: (Noted by Student **During** Supervisory meeting)
- The writing from the papers should be in third person perspective.
- Minimum 10 research papers according to your role.
- Provided Literature Review was not up to standard.
- Added security as my role in the project.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)
- Improve Literature Review
- Continue gathering more Research Papers of various scopes/ sectors.
- Research more on security of the project.

Note: A student MUST attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.

Figure 6.2.2: log sheet 2



Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum TEN (10) for Capstone/FYP Part 1 and TEN (10) for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.

The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Pawan Poudel Group: 19 Date:18/02/2026
Meeting No: 3

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalsina | Supervisor's Signature:

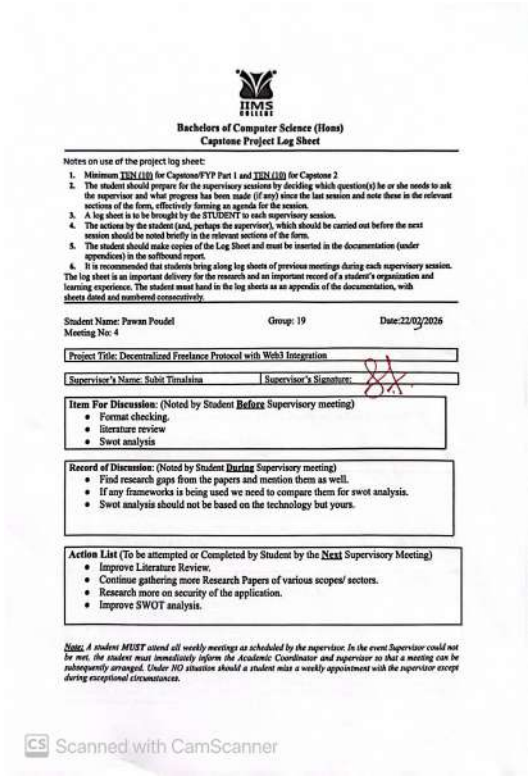
Item For Discussion: (Noted by Student **Before** Supervisory meeting)
- Format checking.
- Literature review.

Record of Discussion: (Noted by Student **During** Supervisory meeting)
- Discussed what frameworks or libraries to use
- Discussed UI/UX
- How to find research papers using keywords.
- Schedule change from two physical meeting to one physical and one online meeting.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)
- Improve Literature Review
- Continue gathering more Research Papers of various scopes/ sectors.
- Research more on security of the application
- Start writing chapter 3 SWOT analysis

Note: A student MUST attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.

Figure 6.2.3: log sheet 3



Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum TEN (10) for Capstone/FYP Part 1 and TEN (10) for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.

The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Pawan Poudel Group: 19 Date:22/02/2026
Meeting No: 4

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalsina | Supervisor's Signature:

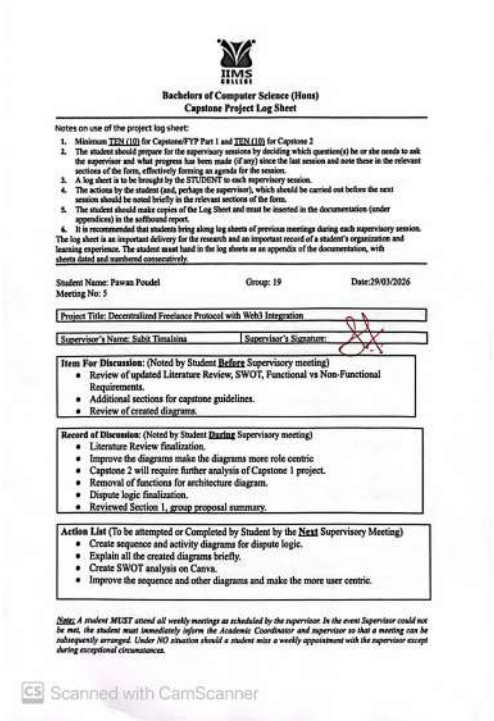
Item For Discussion: (Noted by Student **Before** Supervisory meeting)
- Format checking.
- literature review
- Swot analysis

Record of Discussion: (Noted by Student **During** Supervisory meeting)
- Find research gaps from the papers and mention them as well.
- If any frameworks is being used we need to compare them for swot analysis.
- Swot analysis should not be based on the technology but yours.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)
- Improve Literature Review.
- Continue gathering more Research Papers of various scopes/ sectors.
- Research more on security of the application.
- Improve SWOT analysis.

Note: A student MUST attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.

Figure 6.2.4: log sheet 4



IIMS COLLEGE

Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum TEN (10) for Capstone/FYP Part 1 and TEN (10) for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.

The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Pawan Poudel | Group: 19 | Date:29/03/2026
Meeting No: 5

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalsina | Supervisor's Signature:

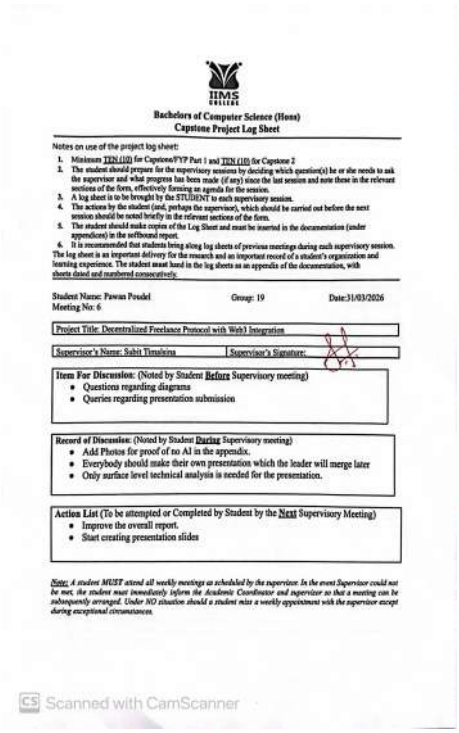
Item For Discussion: (Noted by Student **Before** Supervisory meeting)
- Review of updated Literature Review, SWOT, Functional vs Non-Functional Requirements.
- Additional sections for capstone guidelines.
- Review of created diagrams.

Record of Discussion: (Noted by Student **During** Supervisory meeting)
- Literature Review finalization.
- Improve the diagrams make the diagrams more role centric
- Capstone 2 will require further analysis of Capstone 1 project.
- Removal of functions for architecture diagram.
- Dispute logic finalization.
- Reviewed Section 1, group proposal summary.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)
- Create sequence and activity diagrams for dispute logic.
- Explain all the created diagrams briefly.
- Create SWOT analysis on Canva.
- Improve the sequence and other diagrams and make the more user centric.

Note: *A student MUST attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.*

Figure 6.2.5: log sheet 5



IIMS COLLEGE

Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum TEN (10) for Capstone/FYP Part 1 and TEN (10) for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.

The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Pawan Poudel | Group: 19 | Date:31/03/2026
Meeting No: 6

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalsina | Supervisor's Signature:

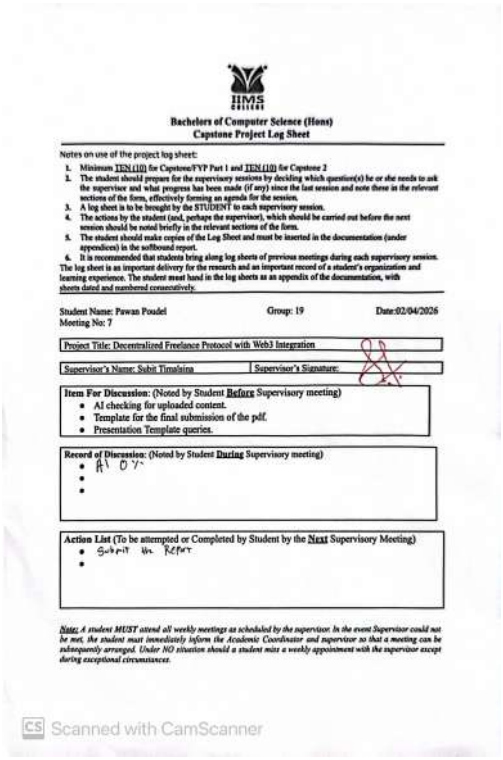
Item For Discussion: (Noted by Student **Before** Supervisory meeting)
- Questions regarding diagrams
- Queries regarding presentation submission

Record of Discussion: (Noted by Student **During** Supervisory meeting)
- Add Photos for proof of no AI in the appendix.
- Everybody should make their own presentation which the leader will merge later
- Only surface level technical analysis is needed for the presentation.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)
- Improve the overall report.
- Start creating presentation slides

Note: *A student MUST attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.*

Figure 6.2.6: log sheet 6



Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum TEN (10) for Capstone/FYP Part 1 and TEN (10) for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session. The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Pawan Poudel Group: 19 Date:02/04/2026
Meeting No: 7

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalsina Supervisor's Signature:

Item For Discussion: (Noted by Student **Before** Supervisory meeting)
- AI checking for uploaded content.
- Template for the final submission of the pdf.
- Presentation Template queries.

Record of Discussion: (Noted by Student **During** Supervisory meeting)
- All OK
-
-

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)
- Submit the Report
-

Note: *A student MUST attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.*

Figure 6.2.7: log sheet 7